

Miniproject evidence

team banana_bread_matcha_latte: marktan, mingyaox, hsuanlec, xinranl3
generated 2026-10-08 15:06 EDT on macOS-26.6.2-arm64-arm-64bit, polars 2.0.0, numpy 2.5.3
project System_Toolchain_mini_project
evidence script sha256 744445fa94498271372f89d7a8053499fd20999c7754842f770eb7bd5f4a157a

Score 10.0 / 10 automatic, of 15 for the miniproject.
REPORT.pdf is worth 5, is submitted separately, and is read by your TA.

What this script found

free	data/tep_fault_free_training.parquet
faulty	data/tep_faulty_training_runs01-20.parquet
pca	results/scores_pca.parquet
ridge	results/scores_ridge.parquet
thresholds	results/thresholds.csv
detection	results/detection.csv
contributions	results/contributions.csv

Detectors 5.0 / 5

[PASS] scores_pca.parquet present
[PASS] every required PCA row scored
100.00% covered, 0 extra rows
[PASS] T2 matches the reference
100.00% of rows within 0.1 %
[PASS] SPE matches the reference
100.00% of rows within 0.1 %
[PASS] scores_ridge.parquet present
[PASS] every required ridge row scored
100.00% covered, 0 extra rows
[PASS] ridge score matches the reference
100.00% of rows within 0.1 %
[PASS] a ridge regression fit in the code

Evaluation and diagnosis 5.0 / 5

[PASS] thresholds.csv with T2, SPE and ridge
[PASS] thresholds are the 99th percentile of the team's validation scores
agree for ['T2', 'SPE', 'ridge']
[PASS] detection.csv matches a recomputation from the team's scores
100% of fault/detector detection rates agree within 0.001
[PASS] false-alarm rows (fault 0) reported for each detector
[PASS] detection.csv covers faults 1 to 20 for all three detectors
[PASS] faults 3, 9 and 15 reported as not detected
largest rate 0.000
[PASS] contributions.csv present
[PASS] top channels match the reference
100% of fault/detector top channels agree

REPORT.pdf 5 points, read by your TA

[SKIP] REPORT.pdf, submitted separately and read by your TA

Transcript

data

tep_fault_free_training.parquet: tep_fault_free_training.parquet, sha256 matches the published file
tep_faulty_training_runs01-20.parquet: tep_faulty_training_runs01-20.parquet, sha256 matches the published file

the script's own detectors

PCA: 31 components reach 90% of the variance
ridge: 2 lags of 52 channels, alpha 1.0

thresholds (99th percentile of validation runs): T2 51.7971, SPE 11.7745, ridge 112.4423
rows to score: 300,000 (validation, test, faults 1-20 runs 1-20)

scores_pca.parquet against the reference

```
{
  "rows_mine": 300000,
  "rows_ref": 300000,
  "covered": 1.0,
  "extra": 0,
  "t2": 1.0,
  "spe": 1.0
}
```

scores_ridge.parquet against the reference

```
{
  "rows_mine": 298800,
  "rows_ref": 298800,
  "covered": 1.0,
  "extra": 0,
  "score": 1.0
}
```

detection.csv against a recomputation

```
100% of fault/detector detection rates agree within 0.001
100% of median delays agree
shape: (12, 7)
????????????????????????????????????????????????????????????????????????????????????
? fault ? detector ? detection_rate ? median_delay_min ? runs_missed ? rate_mine ? delay_mine ?
? --- ? --- ? --- ? --- ? --- ? --- ? --- ?
? i64 ? str ? f64 ? f64 ? i64 ? f64 ? f64 ?
????????????????????????????????????????????????????????????????????????????????????
? 0 ? T2 ? 0.00052 ? null ? null ? 0.00052 ? null ?
? 1 ? T2 ? 0.985417 ? 25.5 ? 0 ? 0.985417 ? 25.5 ?
? 2 ? T2 ? 0.972813 ? 39.0 ? 0 ? 0.972813 ? 39.0 ?
? 3 ? T2 ? 0.000208 ? 231.0 ? 19 ? 0.000208 ? 231.0 ?
? 4 ? T2 ? 0.091354 ? 64.5 ? 0 ? 0.091354 ? 64.5 ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? 7 ? T2 ? 0.995937 ? 9.0 ? 0 ? 0.995937 ? 9.0 ?
? 8 ? T2 ? 0.941875 ? 70.5 ? 0 ? 0.941875 ? 70.5 ?
? 9 ? T2 ? 0.000104 ? 231.0 ? 19 ? 0.000104 ? 231.0 ?
? 10 ? T2 ? 0.084896 ? 471.0 ? 0 ? 0.084896 ? 471.0 ?
? 11 ? T2 ? 0.257708 ? 43.5 ? 0 ? 0.257708 ? 43.5 ?
????????????????????????????????????????????????????????????????????????????????????
```

contributions.csv against the reference

```
34 of 34 top channels agree
fault 1 SPE reference xmeas_4 yours xmeas_4
fault 1 ridge reference xmeas_1 yours xmeas_1
fault 2 SPE reference xmeas_25 yours xmeas_25
fault 2 ridge reference xmeas_10 yours xmeas_10
fault 4 SPE reference xmv_10 yours xmv_10
fault 4 ridge reference xmv_10 yours xmv_10
fault 5 SPE reference xmeas_38 yours xmeas_38
fault 5 ridge reference xmeas_1 yours xmeas_1
fault 6 SPE reference xmv_5 yours xmv_5
fault 6 ridge reference xmeas_1 yours xmeas_1
fault 7 SPE reference xmeas_4 yours xmeas_4
fault 7 ridge reference xmv_4 yours xmv_4
```

the script's detection table, for comparison

```
shape: (21, 4)
????????????????????????????????????????????
```

```

? fault ? T2      ? SPE      ? ridge  ?
? ---   ? ---     ? ---     ? ---   ?
? i64   ? f64     ? f64     ? f64   ?
????????????????????????????????????????
? 0      ? 0.00052 ? 0.0      ? 0.0    ?
? 1      ? 0.985417 ? 0.991875 ? 0.993542 ?
? 2      ? 0.972813 ? 0.958958 ? 0.965104 ?
? 3      ? 0.000208 ? 0.0      ? 0.0    ?
? 4      ? 0.091354 ? 0.994167 ? 0.968854 ?
? ?      ? ?       ? ?       ? ?       ?
? 16     ? 0.019375 ? 0.101146 ? 0.711875 ?
? 17     ? 0.665938 ? 0.8678125 ? 0.866875 ?
? 18     ? 0.870833 ? 0.884375 ? 0.892708 ?
? 19     ? 0.009896 ? 0.003333 ? 0.029167 ?
? 20     ? 0.13625  ? 0.432188 ? 0.635104 ?
????????????????????????????????????????

```

Code: src/miniproject/___init___py

```

"""Tennessee Eastman fault detectors trained on normal data only.

Run the full pipeline with 'uv run miniproject'.
This package does not call the course evidence script.
"""

def main() -> None:
    """Entry point registered in pyproject.toml as 'miniproject'."""
    from miniproject.pipeline import run

    run()

```

Code: src/miniproject/___main___py

```

from miniproject import main

main()

```

Code: src/miniproject/config.py

```

"""Shared choices from the project recipe.

The two detectors are independent, but the assignment says they share this
preprocessing, so it lives in one place:

    "Channels: all 52: xmeas_1 to xmeas_41 and xmv_1 to xmv_11"
    "Training runs: fault-free runs 1 to 300"
    "Validation runs: fault-free runs 301 to 400, used only to set thresholds"
    "Test runs: fault-free runs 401 to 500, used only to measure false alarms"
    "Faulty runs: faults 1 to 20, runs 1 to 20"
    "Standardization: subtract each channel's mean and divide by its standard
    deviation (ddof=1), both computed on the training runs"
"""

from pathlib import Path

# Project root is two levels above this file: src/miniproject/config.py
ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = ROOT / "data"
RESULTS_DIR = ROOT / "results"
FIGURES_DIR = ROOT / "figures"

# "Samples are 3 minutes apart, so a run is 25 hours."
SAMPLE_MINUTES = 3

# "In the faulty file, each fault is introduced one hour into the run, so
# samples 1 to 20 are normal and samples 21 onward are faulty."

```

```
FAULT_START_SAMPLE = 20 # last normal sample; scores after this sample are faulty
```

```
N_CHANNELS_MEAS = 41
```

```
N_CHANNELS_MV = 11
```

```
TRAIN_RUNS = range(1, 301)
```

```
VAL_RUNS = range(301, 401)
```

```
TEST_RUNS = range(401, 501)
```

```
FAULTY_RUNS = range(1, 21)
```

```
FAULTS = range(1, 21)
```

```
# 99th percentile on held-out normal runs. NumPy's default interpolation.
```

```
QUANTILE = 0.99
```

```
# "Keep the smallest number of components k whose eigenvalues add up to
```

```
# at least 90% of the total."
```

```
PCA_VARIANCE = 0.90
```

```
# "Fit one Ridge(alpha=1.0) on the training runs."
```

```
RIDGE_ALPHA = 1.0
```

```
# Detector names as written in the recipe: ('T2', 'SPE', 'ridge').
```

```
DETECTORS = ("T2", "SPE", "ridge")
```

```
CONTRIBUTION_DETECTORS = ("SPE", "ridge")
```

```
# "keeping the top five channels per fault and detector"
```

```
TOP_CHANNELS = 5
```

```
# A fault is treated as caught in the report when more than this share of
```

```
# post-sample-20 rows are in alarm. The alarm rule already suppresses isolated
```

```
# spikes, so 0.10 is well above the roughly 1% threshold rate.
```

```
CAUGHT_RATE = 0.10
```

```
def measurement_names() -> list[str]:
```

```
    """Return the 52 channel names in recipe order."""
```

```
    meas = [f"xmeas_{i}" for i in range(1, N_CHANNELS_MEAS + 1)]
```

```
    mv = [f"xmv_{i}" for i in range(1, N_CHANNELS_MV + 1)]
```

```
    return meas + mv
```

```
# Downs and Vogel (1993), the usual TEP measurement table. Used only so the
```

```
# report can say what a channel is. The model itself uses the raw names.
```

```
CHANNEL_DESCRIPTION: dict[str, str] = {
```

```
    "xmeas_1": "A feed (stream 1)",
```

```
    "xmeas_2": "D feed (stream 2)",
```

```
    "xmeas_3": "E feed (stream 3)",
```

```
    "xmeas_4": "A and C feed (stream 4)",
```

```
    "xmeas_5": "recycle flow (stream 8)",
```

```
    "xmeas_6": "reactor feed rate (stream 6)",
```

```
    "xmeas_7": "reactor pressure",
```

```
    "xmeas_8": "reactor level",
```

```
    "xmeas_9": "reactor temperature",
```

```
    "xmeas_10": "purge rate (stream 9)",
```

```
    "xmeas_11": "product separator temperature",
```

```
    "xmeas_12": "product separator level",
```

```
    "xmeas_13": "product separator pressure",
```

```
    "xmeas_14": "product separator underflow (stream 10)",
```

```
    "xmeas_15": "stripper level",
```

```
    "xmeas_16": "stripper pressure",
```

```
    "xmeas_17": "stripper underflow (stream 11)",
```

```
    "xmeas_18": "stripper temperature",
```

```
    "xmeas_19": "stripper steam flow",
```

```
    "xmeas_20": "compressor work",
```

```
    "xmeas_21": "reactor cooling water outlet temperature",
```

```
    "xmeas_22": "separator cooling water outlet temperature",
```

```
    "xmeas_23": "reactor feed composition A",
```

```

"xmeas_24": "reactor feed composition B",
"xmeas_25": "reactor feed composition C",
"xmeas_26": "reactor feed composition D",
"xmeas_27": "reactor feed composition E",
"xmeas_28": "reactor feed composition F",
"xmeas_29": "purge composition A",
"xmeas_30": "purge composition B",
"xmeas_31": "purge composition C",
"xmeas_32": "purge composition D",
"xmeas_33": "purge composition E",
"xmeas_34": "purge composition F",
"xmeas_35": "purge composition G",
"xmeas_36": "purge composition H",
"xmeas_37": "product composition D",
"xmeas_38": "product composition E",
"xmeas_39": "product composition F",
"xmeas_40": "product composition G",
"xmeas_41": "product composition H",
"xmv_1": "D feed flow valve (stream 2)",
"xmv_2": "E feed flow valve (stream 3)",
"xmv_3": "A feed flow valve (stream 1)",
"xmv_4": "A and C feed flow valve (stream 4)",
"xmv_5": "compressor recycle valve",
"xmv_6": "purge valve (stream 9)",
"xmv_7": "separator liquid flow valve (stream 10)",
"xmv_8": "stripper liquid product valve (stream 11)",
"xmv_9": "stripper steam valve",
"xmv_10": "reactor cooling water flow valve",
"xmv_11": "condenser cooling water flow valve",
}

```

Chiang, Russell and Braatz (2000), Table 1. IDV 1 to 20.

```

FAULT_DESCRIPTION: dict[int, str] = {
    1: "A/C feed ratio, B composition constant (stream 4); step",
    2: "B composition, A/C ratio constant (stream 4); step",
    3: "D feed temperature (stream 2); step",
    4: "reactor cooling water inlet temperature; step",
    5: "condenser cooling water inlet temperature; step",
    6: "A feed loss (stream 1); step",
    7: "C header pressure loss, reduced availability (stream 4); step",
    8: "A, B, C feed composition (stream 4); random variation",
    9: "D feed temperature (stream 2); random variation",
    10: "C feed temperature (stream 4); random variation",
    11: "reactor cooling water inlet temperature; random variation",
    12: "condenser cooling water inlet temperature; random variation",
    13: "reaction kinetics; slow drift",
    14: "reactor cooling water valve; sticking",
    15: "condenser cooling water valve; sticking",
    16: "unknown",
    17: "unknown",
    18: "unknown",
    19: "unknown",
    20: "unknown",
}

```

What a contribution plot should point at if it finds the disturbance
rather than a downstream symptom. The report compares the actual top
channels with this list. Westerhuis, Gurden and Smilde (2000) warn that
contribution plots smear, so a downstream channel can outrank the source.

```

FAULT_EXPECTED_CHANNELS: dict[int, set[str]] = {
    1: {"xmeas_1", "xmeas_4", "xmv_3", "xmv_4", "xmeas_23", "xmeas_25", "xmeas_29", "xmeas_31"},
    2: {"xmeas_4", "xmv_4", "xmeas_24", "xmeas_30"},
    3: {"xmeas_2", "xmv_1"},
    4: {"xmeas_9", "xmeas_21", "xmv_10"},
    5: {"xmeas_11", "xmeas_13", "xmeas_22", "xmv_11"},
    6: {"xmeas_1", "xmv_3"},
    7: {"xmeas_4", "xmv_4", "xmeas_7"},
}

```

```

8: {"xmeas_4", "xmv_4", "xmeas_23", "xmeas_24", "xmeas_25", "xmeas_29", "xmeas_30", "xmeas_31"},
9: {"xmeas_2", "xmv_1"},
10: {"xmeas_4", "xmeas_18", "xmv_4"},
11: {"xmeas_9", "xmeas_21", "xmv_10"},
12: {"xmeas_11", "xmeas_22", "xmv_11"},
13: {"xmeas_7", "xmeas_9", "xmeas_16", "xmeas_18"},
14: {"xmeas_9", "xmeas_21", "xmv_10"},
15: {"xmeas_11", "xmeas_22", "xmv_11"},
16: set(),
17: set(),
18: set(),
19: set(),
20: set(),
}

```

Code: src/miniproject/data.py

```

"""Load the two TEP parquet files and standardize on the training runs.

```

```

Quoted recipe:

```

```

    "Rows to score: every row of the validation, test and faulty runs."
    "Fault-free rows have faultNumber 0."
    "The evidence script refuses files that do not match."

```

```

Training rows are used to fit the models and the scaler. They are not written
into the score files.
"""

```

```

from __future__ import annotations

```

```

import hashlib
from dataclasses import dataclass

```

```

import numpy as np
import pandas as pd

```

```

from miniproject.config import (
    DATA_DIR,
    FAULT_START_SAMPLE,
    measurement_names,
)

```

```

@dataclass

```

```

class PlantData:
    """Standardized blocks plus the run labels needed to score and split them."""

```

```

    channels: list[str]
    mean: np.ndarray
    std: np.ndarray
    # Each block is already standardized with the training mean and std.
    train_z: np.ndarray
    train_meta: pd.DataFrame
    score_z: np.ndarray
    score_meta: pd.DataFrame

```

```

def verify_checksums(data_dir=DATA_DIR) -> None:
    """Refuse the local files if they do not match the published SHA256SUMS.

```

```

    The assignment says the evidence script refuses files that do not match.
    This check runs first so a bad download cannot silently train a model.
    """

```

```

    sums_path = data_dir / "SHA256SUMS"
    if not sums_path.exists():
        raise FileNotFoundError(

```

```

        f"Missing {sums_path}. Download the files in the assignment into data/."
    )
for line in sums_path.read_text().splitlines():
    line = line.strip()
    if not line or line.startswith("#"):
        continue
    digest, name = line.split()
    name = name.lstrip("*")
    path = data_dir / name
    if not path.exists():
        raise FileNotFoundError(path)
    hasher = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1 << 20), b''):
            hasher.update(chunk)
    got = hasher.hexdigest()
    if got != digest:
        raise ValueError(f"Checksum mismatch for {name}: {got} != {digest}")
    print(f"checksum ok {name}")

def _read_parquet(path) -> pd.DataFrame:
    frame = pd.read_parquet(path)
    # Column names in the hosted files are expected to be xmeas_1, xmv_1,
    # simulationRun, sample, and (on the faulty file) faultNumber.
    frame.columns = [str(column) for column in frame.columns]
    return frame

def _require_channels(frame: pd.DataFrame, channels: list[str]) -> None:
    missing = [name for name in channels if name not in frame.columns]
    if missing:
        raise KeyError(
            "Parquet file is missing channels the recipe asks for: "
            + ", ".join(missing[:8])
        )

def _meta(frame: pd.DataFrame, fault_number: np.ndarray | None = None) -> pd.DataFrame:
    """Keep the three identifier columns the score files must contain."""
    if fault_number is None:
        if "faultNumber" not in frame.columns:
            fault_number = np.zeros(len(frame), dtype=np.int64)
        else:
            fault_number = frame["faultNumber"].to_numpy(dtype=np.int64)
    return pd.DataFrame(
        {
            "faultNumber": np.asarray(fault_number, dtype=np.int64),
            "simulationRun": frame["simulationRun"].to_numpy(dtype=np.int64),
            "sample": frame["sample"].to_numpy(dtype=np.int64),
        }
    )

def load_plant(data_dir=DATA_DIR) -> PlantData:
    """Read both files, split the runs, and standardize with training ddof=1.

    Quoted rule: "subtract each channel's mean and divide by its standard
    deviation (ddof=1), both computed on the training runs."

    NumPy's default std uses ddof=0, which would not match the recipe, so the
    divisor is set explicitly.
    """
    channels = measurement_names()
    free = _read_parquet(data_dir / "tep_fault_free_training.parquet")
    faulty = _read_parquet(data_dir / "tep_faulty_training_runs01-20.parquet")
    _require_channels(free, channels)

```

```

_require_channels(faulty, channels)

free_meta = _meta(free)
faulty_meta = _meta(faulty)
if not set(faulty_meta["faultNumber"].unique()).issuperset(set(range(1, 21))):
    raise ValueError("Faulty file does not contain faultNumber 1 to 20.")

# "samples 1 to 20 are normal and samples 21 onward are faulty."
if int(free_meta["sample"].min()) != 1 or int(free_meta["sample"].max()) != 500:
    raise ValueError("Expected fault-free sample numbers 1 to 500.")
if FAULT_START_SAMPLE != 20:
    raise ValueError("Fault injection sample drifted from the recipe.")

free_x = free[channels].to_numpy(dtype=np.float64)
faulty_x = faulty[channels].to_numpy(dtype=np.float64)

train_mask = free_meta["simulationRun"].between(1, 300).to_numpy()
val_mask = free_meta["simulationRun"].between(301, 400).to_numpy()
test_mask = free_meta["simulationRun"].between(401, 500).to_numpy()
if train_mask.sum() == 0 or val_mask.sum() == 0 or test_mask.sum() == 0:
    raise ValueError("Fault-free file is missing training, validation, or test runs.")

# Mean and std come from training runs only. Validation is reserved for
# thresholds, and the test runs are reserved for the false-alarm rate.
train_x = free_x[train_mask]
mean = train_x.mean(axis=0)
std = train_x.std(axis=0, ddof=1)
if np.any(std == 0):
    dead = [channels[i] for i in np.flatnonzero(std == 0)]
    raise ValueError(f"Zero training std for {dead}")

def standardize(values: np.ndarray) -> np.ndarray:
    return (values - mean) / std

train_z = standardize(train_x)
score_z = np.vstack(
    [
        standardize(free_x[val_mask]),
        standardize(free_x[test_mask]),
        standardize(faulty_x),
    ]
)
score_meta = pd.concat(
    [
        free_meta.loc[val_mask],
        free_meta.loc[test_mask],
        faulty_meta,
    ],
    ignore_index=True,
)
# Fault-free scored rows are validation and test. Force faultNumber 0
# even if the source file stored something else.
score_meta.loc[score_meta.index[: val_mask.sum() + test_mask.sum()], "faultNumber"] = 0
score_meta["faultNumber"] = score_meta["faultNumber"].astype(np.int64)

train_meta = free_meta.loc[train_mask].reset_index(drop=True)
train_meta["faultNumber"] = np.int64(0)

print(
    "rows "
    f"train={len(train_meta):,} "
    f"to_score={len(score_meta):,} "
    f"channels={len(channels)}"
)
return PlantData(
    channels=channels,
    mean=mean,

```

```

        std=std,
        train_z=train_z,
        train_meta=train_meta,
        score_z=score_z,
        score_meta=score_meta,
    )

```

Code: src/miniproject/diagnose.py

```

"""Per-channel contributions for SPE and the ridge score.

```

Quoted rule:

```

    "For the SPE and ridge detectors, a statistic that is a sum of squares
    splits naturally into one term per channel. That split is called a
    contribution."

```

```

    "SPE: the contribution of channel j is (z_j - (P P^T z)_j)^2."
    "Ridge: the contribution of channel j is its squared standardized residual."

```

```

    "For each fault and each of the two detectors, average the contributions
    over every sample that is in alarm after sample 20, across the 20 runs.
    ... keeping the top five channels per fault and detector."

```

```

A contribution plot points at where the fault shows up, which is not always
where it started.
"""

```

```

from __future__ import annotations

```

```

import numpy as np
import pandas as pd

```

```

from miniproject.config import FAULTS, TOP_CHANNELS

```

```

def _top_from_average(
    fault: int,
    detector: str,
    average: np.ndarray,
    channels: list[str],
) -> list[dict[str, object]]:
    """Keep the five largest average contributions. Ties break on channel name."""
    order = sorted(
        range(len(channels)),
        key=lambda index: (-float(average[index]), channels[index]),
    )
    rows = []
    for rank, index in enumerate(order[:TOP_CHANNELS], start=1):
        rows.append(
            {
                "fault": fault,
                "detector": detector,
                "rank": rank,
                "channel": channels[index],
                "contribution": float(average[index]),
            }
        )
    return rows

```

```

def spe_contribution_table(
    z: np.ndarray,
    meta: pd.DataFrame,
    reconstructed: np.ndarray,
    alarms: pd.DataFrame,
    channels: list[str],

```

```

) -> pd.DataFrame:
    """Average SPE contributions on post-sample-20 alarming rows."""
    # One term per channel, as specified. Do not renormalize the row.
    contribution = (z - reconstructed) ** 2
    frame = meta.loc[:, ["faultNumber", "simulationRun", "sample"]].copy()
    frame["_row"] = np.arange(len(frame))
    alarm_keys = alarms.loc[alarms["alarm"], ["faultNumber", "simulationRun", "sample"]]
    merged = frame.merge(
        alarm_keys,
        on=["faultNumber", "simulationRun", "sample"],
        how="inner",
    )
    merged = merged[merged["sample"] > 20]

    rows: list[dict[str, object]] = []
    for fault in FAULTS:
        part = merged[merged["faultNumber"] == fault]
        if part.empty:
            print(f"contributions SPE fault {fault}: no alarming samples after sample 20")
            continue
        average = contribution[part["_row"]].to_numpy().mean(axis=0)
        rows.extend(_top_from_average(fault, "SPE", average, channels))
    return pd.DataFrame(
        rows, columns=["fault", "detector", "rank", "channel", "contribution"]
    )

def ridge_contribution_table(
    packed: pd.DataFrame,
    alarms: pd.DataFrame,
    channels: list[str],
) -> pd.DataFrame:
    """Average ridge contributions on post-sample-20 alarming rows.

    'packed' carries one length-52 vector per scored row in '_contrib'.
    """
    alarm_keys = alarms.loc[alarms["alarm"], ["faultNumber", "simulationRun", "sample"]]
    merged = packed.merge(
        alarm_keys,
        on=["faultNumber", "simulationRun", "sample"],
        how="inner",
    )
    merged = merged[merged["sample"] > 20]

    rows: list[dict[str, object]] = []
    for fault in FAULTS:
        part = merged[merged["faultNumber"] == fault]
        if part.empty:
            print(f"contributions ridge fault {fault}: no alarming samples after sample 20")
            continue
        stacked = np.vstack(part["_contrib"].to_list())
        average = stacked.mean(axis=0)
        rows.extend(_top_from_average(fault, "ridge", average, channels))
    return pd.DataFrame(
        rows, columns=["fault", "detector", "rank", "channel", "contribution"]
    )

```

Code: src/miniproject/evaluate.py

"""Thresholds, three-sample alarms, and the detection table.

Quoted rules:

"For each of the three statistics (T2, SPE, ridge), the threshold is
numpy.quantile(scores, 0.99) over the validation runs, with NumPy's
default interpolation."

"A sample is in alarm when it and the two samples before it all exceed the threshold. Alarms never cross a run boundary."

"false-alarm rate: the share of samples in alarm over the test runs, reported as fault 0"

"detection rate: for each faulty run, the share of samples after sample 20 that are in alarm, averaged over the 20 runs"

"detection delay: for each faulty run, minutes from sample 20 to the first alarm after it; report the median over the runs that alarmed, and how many runs never alarmed."

The evaluation checks recompute this table from the team's own scores, so the formulas here are the ones written back to results/detection.csv.
"""

```
from __future__ import annotations
```

```
import numpy as np
import pandas as pd
```

```
from miniproject.config import FAULTS, QUANTILE, SAMPLE_MINUTES
```

```
def alarm_mask(samples: np.ndarray, values: np.ndarray, threshold: float) -> np.ndarray:
    """Mark samples whose own score and the two previous samples all exceed the threshold.
```

```
    "Exceed" is strict: a score equal to the quantile is not an alarm.
    If sample s-1 or s-2 is missing, as with the first two ridge samples,
    that row cannot alarm. The caller must pass a single run.
    """
```

```
    above = values > threshold
    by_sample = {int(sample): bool(flag) for sample, flag in zip(samples, above)}
    return np.fromiter(
        (
            by_sample.get(int(sample), False)
            and by_sample.get(int(sample) - 1, False)
            and by_sample.get(int(sample) - 2, False)
            for sample in samples
        ),
        dtype=bool,
        count=len(samples),
    )
```

```
def attach_alarms(scores: pd.DataFrame, value_col: str, threshold: float) -> pd.DataFrame:
    """Add an alarm column. The window is reset at every (faultNumber, simulationRun)."""
```

```
    parts: list[pd.DataFrame] = []
    grouped = scores.groupby(["faultNumber", "simulationRun"], sort=False)
    for _, part in grouped:
        part = part.sort_values("sample", kind="mergesort").copy()
        part["alarm"] = alarm_mask(
            part["sample"].to_numpy(),
            part[value_col].to_numpy(dtype=np.float64),
            threshold,
        )
        parts.append(part)
    return pd.concat(parts, ignore_index=True)
```

```
def validation_threshold(scores: pd.DataFrame, value_col: str) -> float:
    """99th percentile of fault-free validation runs 301 to 400.
```

```
    np.quantile is called with no method argument so the interpolation is
    whatever NumPy's default is on this interpreter.
    """
```

```

validation = scores[
    (scores["faultNumber"] == 0) & (scores["simulationRun"].between(301, 400))
]
if validation.empty:
    raise ValueError(f"No validation rows for {value_col}.")
return float(np.quantile(validation[value_col].to_numpy(dtype=np.float64), QUANTILE))

def detection_rows(alarmed: pd.DataFrame, detector: str) -> list[dict[str, object]]:
    """One fault-0 row plus faults 1 to 20 for a single detector.

    Fault 0 stores the false-alarm rate in detection_rate. Delay and
    runs_missed are defined only "for each faulty run", so those two cells
    are left empty on the fault-0 row.
    """
    rows: list[dict[str, object]] = []
    test = alarmed[
        (alarmed["faultNumber"] == 0) & (alarmed["simulationRun"].between(401, 500))
    ]
    if test.empty:
        raise ValueError(f"No test rows for {detector}.")
    # "the share of samples in alarm over the test runs"
    rows.append(
        {
            "fault": 0,
            "detector": detector,
            "detection_rate": float(test["alarm"].mean()),
            "median_delay_min": np.nan,
            "runs_missed": np.nan,
        }
    )

    for fault in FAULTS:
        sub = alarmed[alarmed["faultNumber"] == fault]
        runs = sorted(int(run) for run in sub["simulationRun"].unique())
        if len(runs) != 20:
            raise ValueError(f"Fault {fault} has {len(runs)} runs, expected 20.")
        per_run_rate: list[float] = []
        delays: list[float] = []
        missed = 0
        for run in runs:
            part = sub[sub["simulationRun"] == run]
            # "the share of samples after sample 20 that are in alarm"
            after = part[part["sample"] > 20]
            if after.empty:
                raise ValueError(f"Fault {fault} run {run} has no samples after 20.")
            per_run_rate.append(float(after["alarm"].mean()))
            hit = after.loc[after["alarm"], "sample"]
            if hit.empty:
                missed += 1
                continue
            # "minutes from sample 20 to the first alarm after it"
            # Sample 21 is one step (3 minutes) after sample 20.
            first = int(hit.min())
            delays.append(float((first - 20) * SAMPLE_MINUTES))
        rows.append(
            {
                "fault": fault,
                "detector": detector,
                "detection_rate": float(np.mean(per_run_rate)),
                "median_delay_min": float(np.median(delays)) if delays else np.nan,
                "runs_missed": int(missed),
            }
        )
    return rows

```

```

def build_thresholds_and_detection(
    pca_scores: pd.DataFrame,
    ridge_scores: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, pd.DataFrame]]:
    """Thresholds, the detection table, and alarmed frames for diagnosis."""
    specs = (
        ("T2", pca_scores, "T2"),
        ("SPE", pca_scores, "SPE"),
        ("ridge", ridge_scores, "score"),
    )
    threshold_rows = []
    detection: list[dict[str, object]] = []
    alarmed: dict[str, pd.DataFrame] = {}
    for detector, frame, column in specs:
        threshold = validation_threshold(frame, column)
        threshold_rows.append({"detector": detector, "threshold": threshold})
        marked = attach_alarms(frame, column, threshold)
        alarmed[detector] = marked
        detection.extend(detection_rows(marked, detector))
        print(f"threshold {detector:5s} {threshold:.6g}")

    thresholds = pd.DataFrame(threshold_rows, columns=["detector", "threshold"])
    detection_frame = pd.DataFrame(detection)[
        ["fault", "detector", "detection_rate", "median_delay_min", "runs_missed"]
    ]
    return thresholds, detection_frame, alarmed

```

Code: src/miniproject/pca_monitor.py

"""PCA monitor: T2 inside the normal pattern, SPE outside it.

Quoted recipe:

1. Standardize the training data. Compute the principal components of its covariance matrix (on standardized data this is the correlation matrix).
2. Keep the smallest number of components k whose eigenvalues add up to at least 90% of the total.
3. For each standardized row z , with P the k retained loading vectors and λ_i their eigenvalues:

$$t = P^T z$$

$$T^2 = \sum_{i=1}^k t_i^2 / \lambda_i$$

$$SPE = || z - P P^T z ||^2$$

The assignment also says `sklearn.decomposition.PCA` gives the same model: fit on the standardized training data, use `explained_variance_` for λ_i , and `inverse_transform(transform(z))` for $P P^T z$. This file follows that model with `svd_solver="full"`, so the components are the exact eigenvectors rather than a randomized approximation.

T2 asks: "Is it unusual within the normal pattern?"
 SPE asks: "Is it unusual outside the normal pattern?"
 """

```
from __future__ import annotations
```

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
```

```
from miniproject.config import PCA_VARIANCE
```

```
def fit_pca(train_z: np.ndarray) -> tuple[PCA, int]:
    """Fit PCA on standardized training rows and choose k at 90% variance.

    train_z must already be standardized with the training mean and std.
```

Its column mean is then zero, so a second centering inside PCA does not move the loadings. k is the smallest count with cumulative eigenvalue mass at least 0.90. `searchsorted(..., side="left")` implements "at least": an eigenvalue that lands exactly on 90% is kept.

```
"""
if train_z.ndim != 2:
    raise ValueError("train_z must be a 2-d array of standardized rows.")
model = PCA(svd_solver="full").fit(train_z)
cumulative = np.cumsum(model.explained_variance_ratio_)
k = int(np.searchsorted(cumulative, PCA_VARIANCE, side="left") + 1)
if k < 1 or k > train_z.shape[1]:
    raise RuntimeError(f"PCA retained an impossible k={k}.")
# Guard the "smallest k" claim: k-1 components, if any, are not enough.
if k > 1 and cumulative[k - 2] >= PCA_VARIANCE:
    raise RuntimeError("k is not the smallest 90% cutoff.")
print(
    f"PCA k={k} variance={cumulative[k - 1]:.4f} "
    f"of {train_z.shape[1]} channels"
)
return model, k
```

```
def score_pca(model: PCA, k: int, z: np.ndarray, meta: pd.DataFrame) -> pd.DataFrame:
    """Score every supplied row. Caller must pass only validation, test, and faulty rows.
```

Quoted output: results/scores_pca.parquet with columns
faultNumber, simulationRun, sample, T2, SPE.

```
"""
# components_ rows are the loading vectors. P in the recipe is those
# vectors as columns, so P^T z is z @ components_[ :k].T.
components = model.components_[ :k]
lam = model.explained_variance_[ :k]
if np.any(lam <= 0):
    raise RuntimeError("A retained PCA eigenvalue is not positive.")

# Training rows were centered before PCA, and PCA subtracts that mean
# again inside transform. The recipe's z is the standardized row, whose
# training mean is zero. Project z directly so SPE matches
# ||z - P P^T z||^2 and not a second recentering.
scores = z @ components.T
reconstructed = scores @ components
t2 = np.sum(scores**2 / lam, axis=1)
spe = np.sum((z - reconstructed) ** 2, axis=1)

out = meta.loc[:, ["faultNumber", "simulationRun", "sample"]].copy()
out["T2"] = t2
out["SPE"] = spe
out = out.sort_values(["faultNumber", "simulationRun", "sample"], kind="mergesort")
out = out.reset_index(drop=True)
return out
```

```
def reconstruction(model: PCA, k: int, z: np.ndarray) -> np.ndarray:
    """Return P P^T z for contribution plots.
```

```
"""
SPE contribution of channel j is (z_j - (P P^T z)_j)^2.
components = model.components_[ :k]
scores = z @ components.T
return scores @ components
```

Code: `src/miniproject/pipeline.py`

```
"""Run the whole project except the evidence script.
```

The recipe, in order:

1. Check SHA256 of the files in data/.
2. Standardize with the training mean and std (ddof=1) from fault-free runs 1-300.
3. Score every validation, test, and faulty row.
4. Set each threshold with numpy.quantile(validation scores, 0.99).
5. Alarm only when a sample and the two samples before it all exceed the threshold.
6. Write the detection table, the contribution table, the figure, and REPORT.pdf.

Quoted outputs:

```

results/scores_pca.parquet
results/scores_ridge.parquet
results/thresholds.csv
results/detection.csv
results/contributions.csv
REPORT.pdf
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from miniproject.config import RESULTS_DIR
from miniproject.data import load_plant, verify_checksums
from miniproject.diagnose import ridge_contribution_table, spe_contribution_table
from miniproject.evaluate import (
    alarm_mask,
    build_thresholds_and_detection,
    validation_threshold,
)
from miniproject.pca_monitor import fit_pca, reconstruction, score_pca
from miniproject.plots import plot_fault_run
from miniproject.report import pick_plot_fault, write_report
from miniproject.ridge_monitor import fit_ridge, score_ridge

def _check_alarm_rule() -> None:
    """The three-sample rule, including a missing sample that must not alarm."""
    dense = alarm_mask(
        np.array([1, 2, 3, 4]),
        np.array([5.0, 5.0, 5.0, 0.0]),
        1.0,
    )
    if dense.tolist() != [False, False, True, False]:
        raise RuntimeError("Alarm rule failed on a complete run.")
    # Sample 2 is absent, so sample 4 does not have two preceding scores.
    sparse = alarm_mask(
        np.array([1, 3, 4]),
        np.array([5.0, 5.0, 5.0]),
        1.0,
    )
    if sparse.tolist() != [False, False, False]:
        raise RuntimeError("Alarm rule crossed a missing sample.")

def _write_csv(frame: pd.DataFrame, path) -> None:
    """Enough digits that a later read-back matches the in-memory floats."""
    frame.to_csv(path, index=False, float_format="%.17g")

def _cast_keys(frame: pd.DataFrame) -> pd.DataFrame:
    out = frame.copy()
    for column in ("FaultNumber", "simulationRun", "sample"):
        out[column] = out[column].astype("int64")
    return out

```

```

def _assert_score_rows(pca_scores: pd.DataFrame, ridge_scores: pd.DataFrame) -> None:
    """Every required row scored, and no others."""
    if len(pca_scores) != 300_000:
        raise RuntimeError(f"PCA score file has {len(pca_scores)} rows, expected 300000.")
    # Each of 600 scored runs drops its first two samples.
    if len(ridge_scores) != 298_800:
        raise RuntimeError(f"Ridge score file has {len(ridge_scores)} rows, expected 298800.")
    free = pca_scores[pca_scores["faultNumber"] == 0]
    if int(free["simulationRun"].min()) != 301 or int(free["simulationRun"].max()) != 500:
        raise RuntimeError("Fault-free PCA scores are not runs 301 to 500.")
    if set(ridge_scores["sample"].unique()).issuperset({1, 2}):
        raise RuntimeError("Ridge scores include sample 1 or 2, which have no t-2.")
    for frame, name in ((pca_scores, "PCA"), (ridge_scores, "ridge")):
        if frame.duplicated(["faultNumber", "simulationRun", "sample"]).any():
            raise RuntimeError(f"{name} scores have duplicate keys.")

def _assert_tables_match_scores(
    pca_scores: pd.DataFrame,
    ridge_scores: pd.DataFrame,
    thresholds: pd.DataFrame,
    detection: pd.DataFrame,
) -> None:
    """Recompute the 99th percentiles from the score frames about to be saved.

    The evaluation check does this from the team's own scores, so a threshold
    that is not that percentile would fail even when the detector is right.
    """
    expected = {
        "T2": validation_threshold(pca_scores, "T2"),
        "SPE": validation_threshold(pca_scores, "SPE"),
        "ridge": validation_threshold(ridge_scores, "score"),
    }
    for _, row in thresholds.iterrows():
        got = float(row["threshold"])
        if got != expected[row["detector"]]:
            raise RuntimeError(f"Threshold for {row['detector']} does not match quantile.")
    if len(detection) != 63:
        raise RuntimeError("detection.csv must have 21 rows for each of 3 detectors.")
    faults = set(detection["fault"].astype(int))
    if faults != set(range(0, 21)):
        raise RuntimeError("detection.csv is missing a fault from 0 to 20.")

def run() -> None:
    """Fit, score, evaluate, diagnose, and write the report.

    Does not call the course evidence script.
    """
    _check_alarm_rule()
    verify_checksums()
    plant = load_plant()

    model, k = fit_pca(plant.train_z)
    cumulative = np.cumsum(model.explained_variance_ratio_)
    variance = float(cumulative[k - 1])
    pca_scores = _cast_keys(score_pca(model, k, plant.score_z, plant.score_meta))
    reconstructed = reconstruction(model, k, plant.score_z)

    ridge_model, resid_std = fit_ridge(plant.train_z, plant.train_meta)
    # The recipe checks that the code contains a ridge fit. This is that fit.
    if not hasattr(ridge_model, "coef_"):
        raise RuntimeError("Ridge model was not fit.")
    ridge_scores, ridge_contrib = score_ridge(
        ridge_model, resid_std, plant.score_z, plant.score_meta
    )
    ridge_scores = _cast_keys(ridge_scores)

```

```

_assert_score_rows(pca_scores, ridge_scores)

thresholds, detection, alarmed = build_thresholds_and_detection(pca_scores, ridge_scores)
_assert_tables_match_scores(pca_scores, ridge_scores, thresholds, detection)

spe_table = spe_contribution_table(
    plant.score_z,
    plant.score_meta,
    reconstructed,
    alarmed["SPE"],
    plant.channels,
)
ridge_table = ridge_contribution_table(ridge_contrib, alarmed["ridge"], plant.channels)
contributions = pd.concat([spe_table, ridge_table], ignore_index=True)
contributions = contributions.sort_values(
    ["detector", "fault", "rank"], kind="mergesort"
).reset_index(drop=True)

RESULTS_DIR.mkdir(parents=True, exist_ok=True)
pca_path = RESULTS_DIR / "scores_pca.parquet"
ridge_path = RESULTS_DIR / "scores_ridge.parquet"
pca_scores.to_parquet(pca_path, index=False)
ridge_scores.to_parquet(ridge_path, index=False)
_write_csv(thresholds, RESULTS_DIR / "thresholds.csv")
_write_csv(detection, RESULTS_DIR / "detection.csv")
_write_csv(contributions, RESULTS_DIR / "contributions.csv")

# Read the parquet back. This is our check, not the course evidence script.
reloaded = pd.read_parquet(pca_path)
if not np.allclose(reloaded["T2"], pca_scores["T2"], rtol=0, atol=0):
    raise RuntimeError("PCA parquet did not round-trip.")

figure = plot_fault_run(
    pca_scores,
    ridge_scores,
    dict(zip(thresholds["detector"], thresholds["threshold"])),
    fault=pick_plot_fault(detection),
    run=1,
)
write_report(
    detection=detection,
    thresholds=thresholds,
    contributions=contributions,
    figure=figure,
    k=k,
    variance=variance,
    pca_scores=pca_scores,
    ridge_scores=ridge_scores,
)
_print_summary(detection, k, variance)

def _print_summary(detection: pd.DataFrame, k: int, variance: float) -> None:
    print(f"\nPCA k={k} retained variance={variance:.4f}")
    print("fault T2_rate SPE_rate ridge_rate T2_miss SPE_miss ridge_miss")
    for fault in range(0, 21):
        cells = []
        misses = []
        for detector in ("T2", "SPE", "ridge"):
            row = detection[
                (detection["fault"] == fault) & (detection["detector"] == detector)
            ].iloc[0]
            cells.append(f"{row['detection_rate']:.3f}")
            missed = row["runs_missed"]
            misses.append(f" .{int(pd.isna(missed))} else {int(missed):3d}")
        print(
            f"{fault:5d} {cells[0]:>7} {cells[1]:>8} {cells[2]:>10} "

```

```
        f"{misses[0]} {misses[1]} {misses[2]}"
    )

if __name__ == "__main__":
    run()
```

Summary line

```
{"team": "banana_bread_matcha_latte", "andrew_ids": ["marktan", "mingyaox", "hsuanlec", "xinranl3"],
 "generated": "2026-10-08 15:06 EDT", "auto_score": 10.0, "auto_of": 10, "held_for_ta": 0.0, "total":
 15, "script_sha256": "744445fa94498271372f89d7a8053499fd20999c7754842f770eb7bd5f4a157a"}
```